

Human Development Index (HDI) Predictor

Project Description

The **Human Development Index (HDI) Predictor** is an end-to-end Machine Learning web application designed to instantly evaluate the human development tier of a hypothetical region or actual country based on core socioeconomic indicators.

Created to reflect the United Nations Development Programme (UNDP) calculations, the platform predicts HDI using four metrics: Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) Per Capita. It features an interactive **Country Autofill** module that allows users to instantly populate real-world historical data for over 200 countries, making predictive testing accessible and educational.

Built with Flask and powered by a highly accurate Scikit-Learn Multiple Linear Regression model, the platform delivers instant analytics wrapped in a modern, responsive, and mobile-friendly UI.

Scenarios

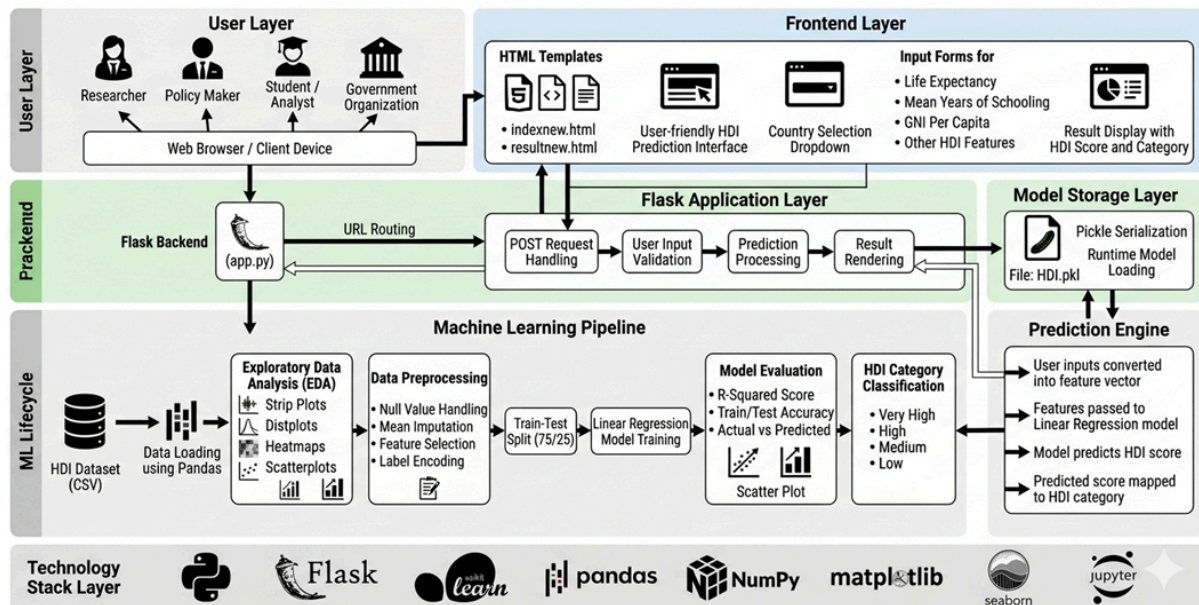
Scenario 1: A user wants to see how a specific real-world country performs. They select "India" from the autofill dropdown. The system automatically fetches India's latest historical data for life expectancy, schooling, and income, fills the form, and instantly predicts its HDI tier.

Scenario 2: A policy maker is testing hypothetical scenarios. They manually enter values for a hypothetical region (e.g., increasing the mean years of schooling by 5 years) to evaluate how focused educational investments could theoretically push a region from a "Medium" to "High" HDI category.

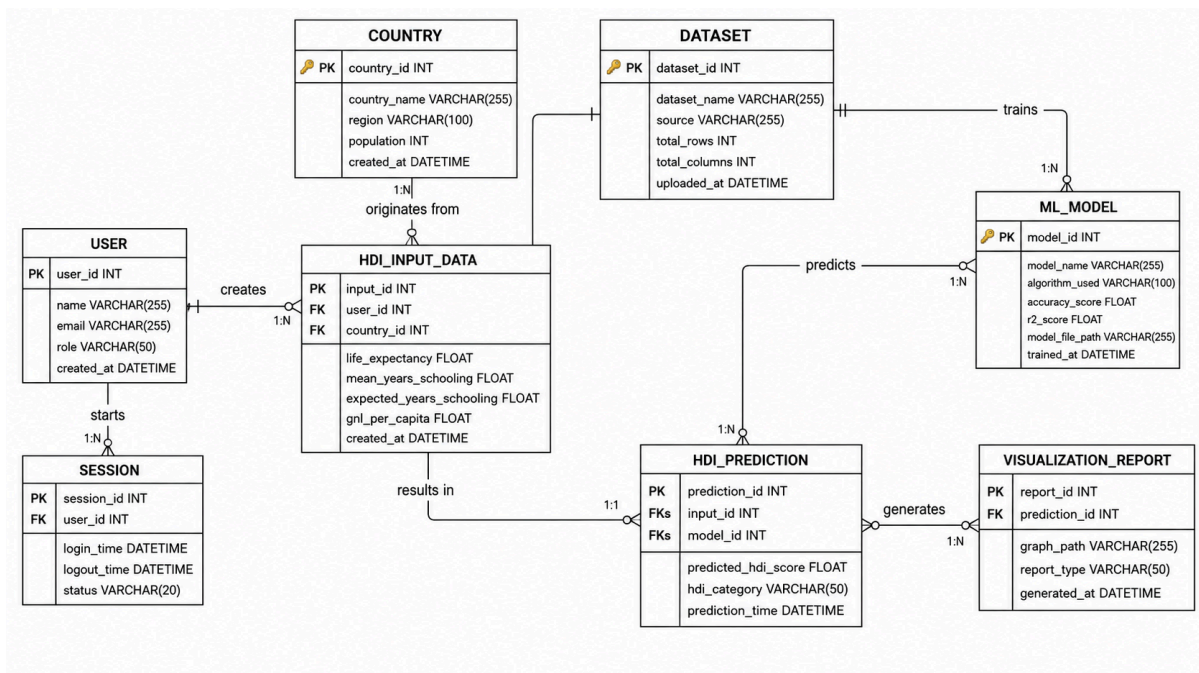
Scenario 3: A data science student uses the tool to understand how the UN handles wealth skewness. By adjusting the GNI per capita input drastically, they can observe how the backend's logarithmic transformation of income affects the final index score logarithmically rather than linearly.

Technical Architecture

Description: The HDI Predictor utilizes a modular three-tier architecture. The frontend provides a responsive user interface built with HTML, CSS, and Bootstrap 5. The Flask-based backend orchestrates data validation, performs necessary mathematical transformations (like $\ln$ on GNI), and interfaces with a serialized Scikit-Learn Multiple Linear Regression model (HDI.pkl) to generate the prediction.



ER Diagram



Pre-requisites

- **Python Programming Proficiency:** [Python Documentation](#)
- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning (Scikit-Learn):** [Scikit-Learn Documentation](#)
- **Data Manipulation (Pandas & Numpy):** [Pandas Docs](#)
- **HTML, CSS, and Bootstrap 5 Skills:** [Bootstrap Documentation](#)
- **Version Control with Git:** [Git Documentation](#)

Project Workflow

Milestone 1: Data Acquisition and Preprocessing

- **Activity 1.1:** Gather historical HDI datasets from Kaggle and official UNDP reports.
- **Activity 1.2:** Perform Exploratory Data Analysis (EDA) to handle missing values and validate feature correlations.
- **Activity 1.3:** Clean the data and export it as hdi_cleaned.csv for both model training and frontend usage.

Milestone 2: Model Training and Serialization

- **Activity 2.1:** Implement a logarithmic transformation pipeline for the Gross National Income (GNI) feature to normalize wealth distribution.
- **Activity 2.2:** Train a Multiple Linear Regression model using Scikit-Learn to map the four features to the final HDI score.
- **Activity 2.3:** Serialize and export the trained model as a .pkl file for integration into the web server.

Milestone 3: Backend Development (Flask)

- **Activity 3.1:** Write the main application logic in app.py, establishing routing for the Home, About, Portal, and Predict pages.
- **Activity 3.2:** Develop the /predict endpoint to process form submissions, apply the log transformation to incoming data, and pass it to the loaded ML model for inference.

```

1 @app.route('/predict', methods=['GET', 'POST'])
2 def predict():
3     if request.method == 'POST':
4         try:
5             # Get data from form
6             life_expectancy = float(request.form['life_expectancy'])
7             expec_yr_school = float(request.form['expec_yr_school'])
8             mean_yr_school = float(request.form['mean_yr_school'])
9             gross_inc_percap = float(request.form['gross_inc_percap'])
10            # Preprocess (log transform GNI per capita)
11            log_gross_inc_percap = np.log(gross_inc_percap) if gross_inc_percap > 0 else 0
12            # Predict
13            features = np.array([[life_expectancy, expec_yr_school, mean_yr_school, log_gross_inc_percap]])
14            prediction = model.predict(features)[0] if model else 0.0
15            prediction = min(max(prediction, 0.0), 1.0)
16            category, _ = get_hdi_category(prediction)
17            return render_template('result.html', prediction=round(prediction, 3), category=category)
18        except Exception as e:
19            return render_template('predict.html', error=str(e))

```

- **Activity 3.3:** Build a Python extraction script to generate a static country_data.json file to serve fast, database-free data to the frontend.

Milestone 4: Frontend Development

- **Activity 4.1:** Design a modern, responsive UI using Bootstrap 5, featuring visually appealing bento-grid layouts.
- **Activity 4.2:** Implement the Country Autofill feature using vanilla JavaScript and the Fetch API to dynamically populate input fields based on dropdown selection.

```

1 document.addEventListener('DOMContentLoaded', function() {
2     const countrySelect = document.getElementById('country_select');
3     fetch("/static/data/country_data.json")
4         .then(res => res.json())
5         .then(data => {
6             Object.keys(data).sort().forEach(c => {
7                 const opt = new Option(c, c);
8                 countrySelect.add(opt);
9             });
10        });
11    countrySelect.onchange = function() {
12        const d = countryData[this.value];
13        if (d) {
14            document.getElementById('life_expectancy').value = d.life_expectancy;
15            // ... fill other fields
16        }
17    };
18 });

```

- **Activity 4.3:** Create dynamic results rendering using Jinja templating to display color-coded badges based on the predicted score.

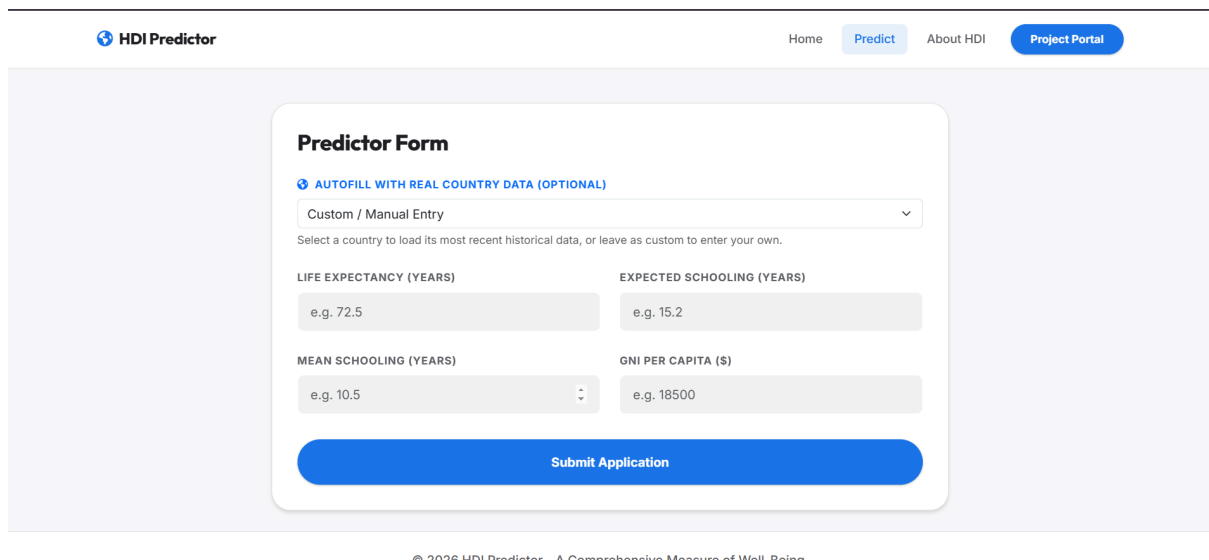
Milestone 5: Deployment Preparation

- **Activity 5.1:** Set up a requirements.txt file detailing all necessary dependencies.
- **Activity 5.2:** Configure a Procfile for seamless cloud deployment using Gunicorn.

Exploring the Website's Web Pages

Predictor Page (Input Section)

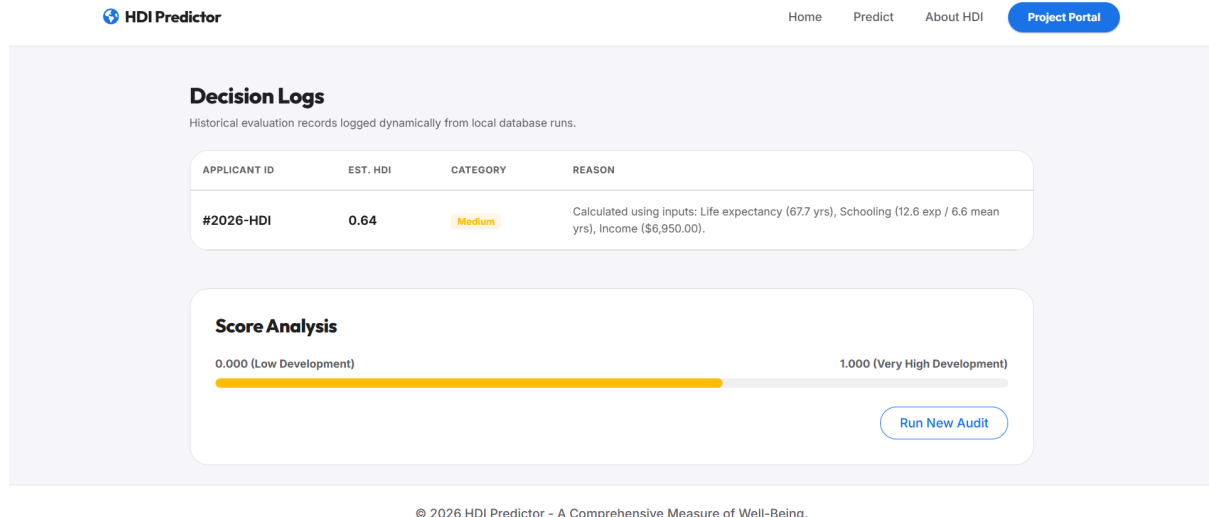
Description: A clean, card-based interface containing the main generation form. It features a dropdown menu for autofill and manual fields for socioeconomic metrics.



The screenshot shows the 'HDI Predictor' website interface. At the top, there is a navigation bar with links for 'Home', 'Predict', 'About HDI', and 'Project Portal'. The main content area features a 'Predictor Form' card. The form has a title 'Predictor Form' and a link 'AUTOFILL WITH REAL COUNTRY DATA (OPTIONAL)'. Below this is a dropdown menu currently set to 'Custom / Manual Entry'. A note states: 'Select a country to load its most recent historical data, or leave as custom to enter your own.' The form contains four input fields: 'LIFE EXPECTANCY (YEARS)' with a placeholder 'e.g. 72.5', 'EXPECTED SCHOOLING (YEARS)' with a placeholder 'e.g. 15.2', 'MEAN SCHOOLING (YEARS)' with a placeholder 'e.g. 10.5' and a small dropdown arrow, and 'GNI PER CAPITA (\$)' with a placeholder 'e.g. 18500'. At the bottom of the form is a large blue button labeled 'Submit Application'. The footer of the page reads '© 2026 HDI Predictor - A Comprehensive Measure of Well-Being.'

Predictor Page (Results Section)

Description: Displays the AI's prediction as a decimal score and a color-coded development tier badge.



© 2026 HDI Predictor - A Comprehensive Measure of Well-Being.

Conclusion

The Human Development Index Predictor is a comprehensive, full-stack machine learning application that bridges complex statistical data and user-friendly web interfaces. By combining accurate models with Flask and Bootstrap, it provides educational insights into global metrics. Future plans include live API integration and interactive maps.